Final Reflection - Project ARISS

Autonomous Robotic Inspection and Sampling System

Jayden Hazell - 24953364

Robotics Studio 2

University of Technology Sydney

Faculty of Engineering and IT

June 11, 2026

Contents

Introduction

This document presents a final reflection on the development of the ARISS (Autonomous Robotic Inspection and Sampling System) project and the supporting artefacts produced throughout the semester. ARISS was developed as an autonomous robotic scanning system designed to generate 3D models of objects using a UR3e robot and a RealSense RGB-D camera, with the aim of reducing human contact with sensitive or hazardous materials.

The project required the analysis of a complex robotics problem as an interconnected system rather than as a set of isolated tasks. Its architecture combined perception and mapping, motion planning and control, interaction and execution, and model generation within a coordinated state-machine framework, with the main control layer acting as the supervisory backbone that managed sequencing, subsystem communication, execution flow, and fault handling across the platform.

This reflection evaluates how the system developed across the sprint cycle and how the artefacts included in this report demonstrate the progression from early subsystem implementation toward a more integrated and technically mature robotics solution. In doing so, the document considers not only the technical development of the project, but also the collaborative processes, system-level decision making, and critical evaluation required to build, test, document, and refine a working robotic platform in a constrained studio environment.

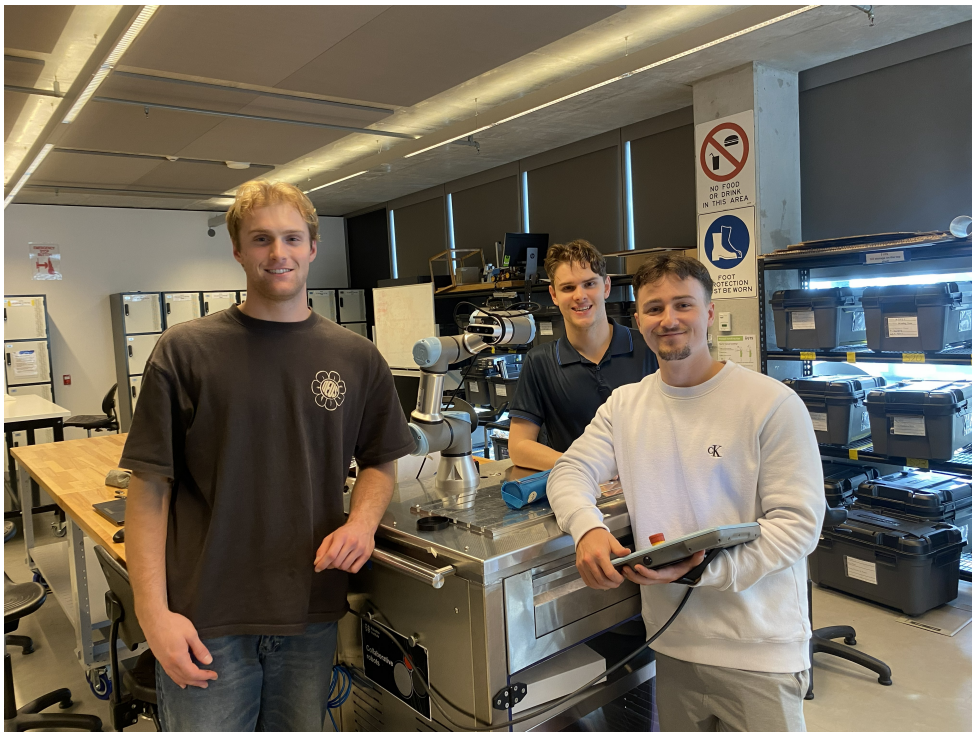


Figure 1: Group Photo with the ARISS Robot

Sprint 1 — Planning and Experimentation

Sprint 1 established the foundations the entire project would be built on: a shared understanding of the problem, a clear decomposition of responsibilities, and an honest assessment of the risks involved in operating a collaborative robot in a shared lab environment. Before any code was written or hardware was touched, the team needed to agree on what the system had to do, who owned each piece of it, and what could go wrong. The three artefacts produced in this sprint: the project contract, the risk assessment, and the system architecture diagram which collectively answered those questions and provided a reference point that shaped every subsequent sprint.

Artefact 1.1 — Project Contract

The project contract was a collaborative document that formalised the scope, objectives, and measurable success criteria for ARISS. Tomas led the drafting of the document while the team worked together on the overall deliverables sections. My primary contribution was authoring the Perception and Mapping subsystem evaluation criteria, which defined what the subsystem was required to achieve at each grade band from Pass through to High Distinction. Defining these criteria before development began was an act of systems thinking: it forced me to reason about the subsystem not as a list of code to write, but as a set of outputs that had to be produced, measurable thresholds that had to be met, and interfaces that had to be respected. Table 1 summarises the criteria I defined for the Perception and Mapping subsystem.

	Key capability	**Measurable outcome**
P	Reliable data acquisition and basic boundary detection	RGB-D feed live in GUI; known starting pose in RViz; basic extrinsic alignment of UR3e and camera frames; points placed in global frame.
C	Multi-view point cloud with basic reconstruction	Registered point cloud from multiple viewpoints; noise filtering applied; coarse 3D model produced; exportable point cloud file.
D	Clean object isolation and watertight mesh	Multi-view registration; robust outlier rejection; object segmented from workspace; watertight surface mesh; mesh preview in GUI; scan coverage metric reported.
HD	Adaptive, high-quality reconstruction	High-resolution dense reconstruction; incomplete surface regions detected; occluded areas identified; quality diagnostics published; stable global alignment maintained.
P	Fully autonomous, self-assessing reconstruction	CAD-ready mesh with topology refinement and hole filling; incomplete regions flagged for autonomous rescanning; real-time diagnostics reported; automated quality assessment generated.

Table 1: Perception and Mapping subsystem evaluation criteria as defined in the project contract (Pass–Perfect).

At the time of writing, the team believed Distinction to High Distinction was well within reach. The system D-level criteria included adaptive rescanning—detecting low-density regions, performing targeted rescans, and merging the results into a consolidated mesh. As the semester progressed it became clear this was significantly more complex than anticipated.

Adaptive rescanning required all three subsystems to be operating reliably in concert before gap-filling logic could even be attempted, and the reconstruction pipeline had been developed as a standalone offline script (`reconstruct.py`). Integrating it into the live ROS2 system within the remaining time was not feasible.

The team replaced adaptive rescanning at the system D-level with advanced safety—a heartbeat watchdog detecting node failures within three seconds, a boot sequence validating camera, TF tree, and joint files before transitioning to `IDLE`, and a latched emergency stop requiring a full system restart. This was a deliberate engineering judgement: delivering robust, demonstrable safety behaviour rather than a shallow implementation of a complex feature. The individual subsystem criteria were not modified; the change was at the system evaluation level only.

Artefact 1.2 — Risk Assessment

The risk assessment was led by Lachlan, with Tomas and I reviewing and signing off on the final document. Rather than treating it as a compliance exercise, the team drew on prior experience across lab subjects to identify the specific hazards associated with operating a UR3e in a shared student environment—electrical handling, unexpected robot motion, payload exceedance, and operator fatigue—and to define proportionate controls for each.

Following submission, feedback was received from the lab supervisor:

> *"Hi team, residual risk should be LOW for all tasks after control measures. Please fix this in your Risk Assessment and send it to me on Teams — Tony."*

The three tasks where residual risk had been left at Medium or High were mishandling electrical cables and devices, electrocution, and fire in the lab. In response, the team reviewed the control measures for each and refined them to sufficiently reduce the residual risk to Low. For electrical cable handling this included adding pre-work inspection procedures and visible cable covers; for electrocution and fire, exclusion zones around hazardous devices and regular checks of RCD circuit breakers and emergency stops were formalised as required controls. The updated document was resubmitted to the supervisor. This was a straightforward but important correction — ensuring the risk assessment reflected a genuinely safe working environment rather than one that looked compliant on paper.

Artefact 1.3 — System Architecture Diagram

I produced the system architecture diagram to give the team a shared visual language for how the three subsystems—Perception and Mapping, Motion Planning and Control, and Interaction and Execution—would communicate. The diagram showed the major nodes, the hardware components they interfaced with, and the data flowing between them, using a colour-coded legend to distinguish hardware, nodes, processes, GUI inputs, GUI outputs, and data objects.

The original diagram identifies the three subsystems, their internal processes, and hardware interfaces. Connections between subsystems are shown but carry no information about the underlying ROS2 topics or message types used for communication.

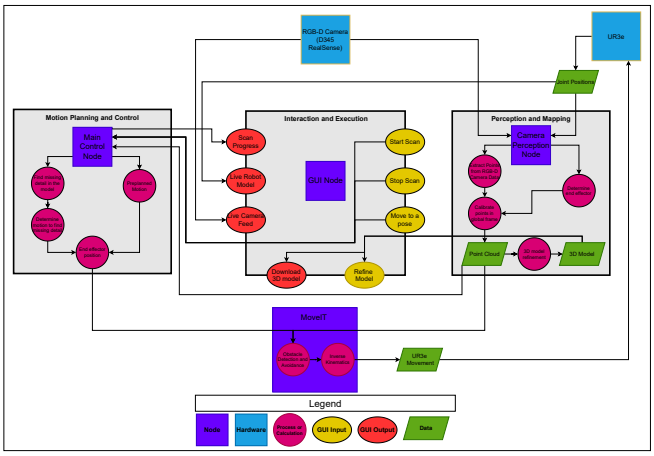


Figure 2: Original System Architecture Diagram

The Sprint 1 diagram received the following feedback:

> *"Covers major subsystems clearly. Missing detail in interface definitions, e.g. ROS message types."*

The revised diagram retains the same structure and layout but annotates each interface connection with its ROS2 topic name and message type This directly addresses the feedback that interface definitions lacked sufficient detail. Note that `main_control_node` was introduced in Sprint 3 and is therefore not included in the revised diagram.

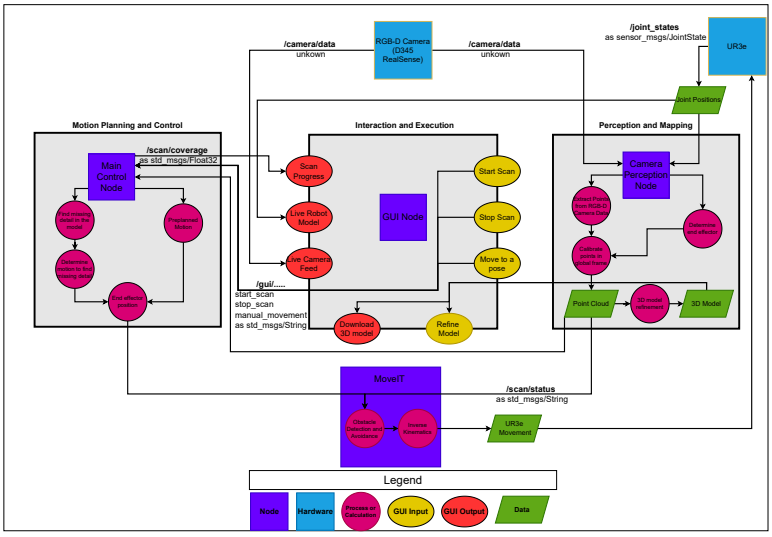


Figure 3: Updated System Architecture Diagram

Sprint 2 — Subsystem Development

Sprint 2 focused on building the perceptual foundation the rest of the system depended on. Before any point cloud registration or mesh generation could be attempted, three things had to be true: the camera and robot had to share a common coordinate frame, the system had to validate itself before accepting commands, and the data captured at each viewpoint had to be trustworthy enough to reconstruct from. This sprint addressed all three.

The formal tests defined in the Sprint 2 presentation were replaced with more foundational validations, as the originals were focused on later pipeline stages that had not yet been reached.

Artefact 2.1 — Hand-Eye Calibration

Before scanning could begin, the fixed spatial relationship between the robot wrist (`tool0`) and the camera body (`d435i_link`) had to be precisely determined. This transform cannot be measured directly with a ruler — any small error in the rotation component compounds across the robot's reach, shifting where the system believes the camera is in 3D space. The calibration was performed using a ChArUco board: a hybrid target combining a chessboard with embedded ArUco fiducial markers, which allows subpixel corner localisation even when the board is partially occluded or viewed at an extreme angle.

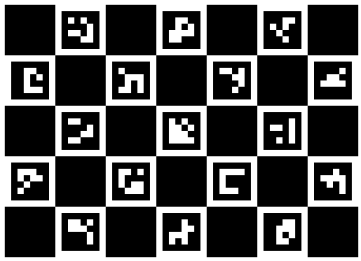


Figure 4: ChArUco calibration board

Twenty-two paired observations were collected — each pairing the board pose in the camera frame with the robot end-effector pose in the world frame — across diverse wrist orientations and camera tilt angles. Five solvers were run against these samples (TSAI, PARK, HORAUD, ANDREFF, DANIILIDIS). The final transform was computed as the geodesic mean of the three agreeing solvers. Table 2 shows the calibrated result.

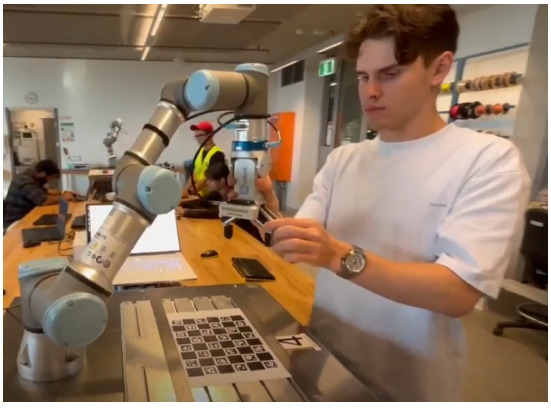


Figure 5: Collecting calibration samples

tx (m)	ty (m)	tz (m)	qx	qy	qz	qw
-0.022589	0.010380	0.051146	0.510643	-0.493535	0.493614	0.502009

Table 2: Static hand-eye transform parameters (`tool0` → `d435i_link`) as deployed in `master.launch.py`.

To validate the calibration, the dimensions of the test artefact were compared against the CAD reference measurements shown in Figure 6. While this confirmed the transform was roughly correct, it could not guarantee calibration success — it was unclear which points within the transform tree the solver was measuring from. Final confirmation was deferred to the perception pipeline in later sprints, where the reconstructed output could be assessed directly against the known geometry.

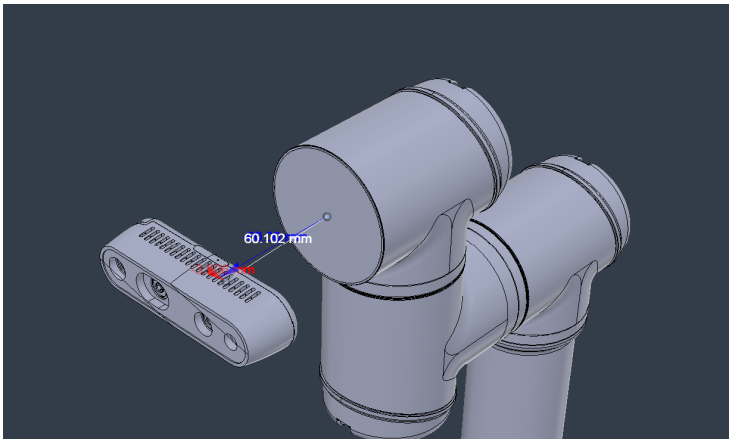


Figure 6: CAD reference measurements of the camera relative to the end effector

An earlier calibration attempt, using insufficient samples and captures taken while the robot was still in motion, produced a transform offset by only a few centimetres from the CAD reference. Despite appearing minor, this error caused severe ghosting in the reconstruction output — as seen in Figure 7, the object appears doubled as captures from different viewpoints disagreed on its position in the global frame. Figure 8 shows the clean result after recalibration, confirming that even small calibration errors compound into unusable reconstructions.

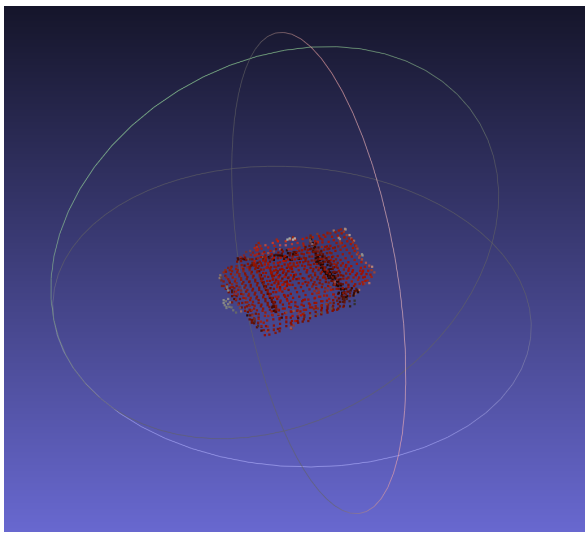


Figure 7: Early reconstruction showing ghosting

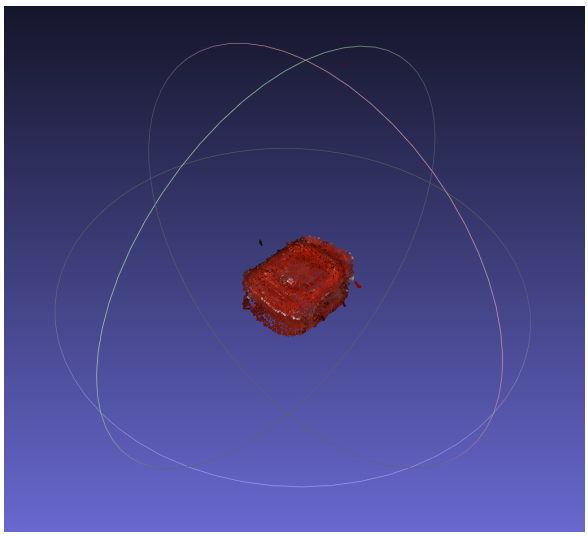


Figure 8: Correct reconstruction after recalibration

Artefact 2.2 — Scan Session Data Structure

Each scan session was structured to make every capture independently verifiable. At the session level, `session.json` recorded the coordinate frame identifiers and a snapshot of the calibration transforms taken at boot time. At the capture level, `capture.json` recorded everything needed to reconstruct the camera pose and assess data quality for that individual viewpoint. Figure 9 shows the session directory structure produced after a complete scan.

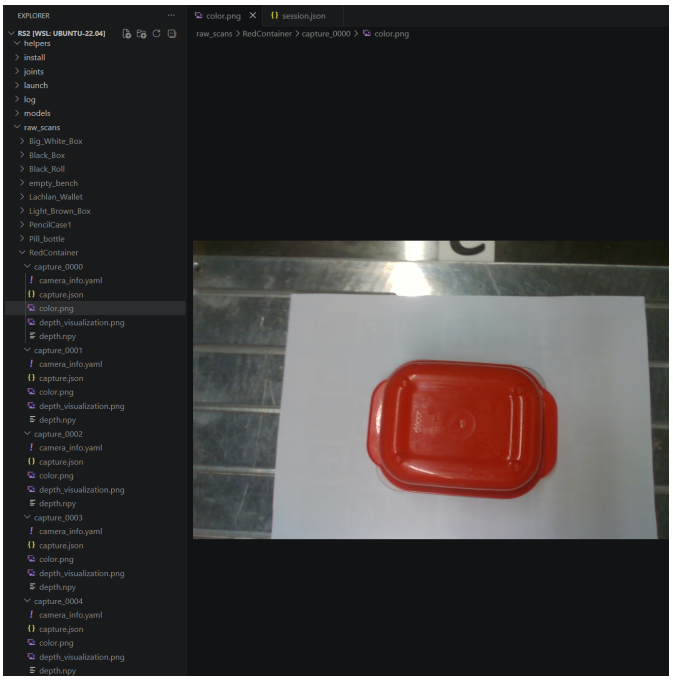


Figure 9: Scan session directory structure

File	Format	Trigger
`{session_dir}/session.json`	JSON	First `SCAN` command in a new session
`{capture_dir}/color.png`	PNG	Each successful capture
`{capture_dir}/depth.npy`	NumPy uint16	Each successful capture if `SAVE_DEPTH_NPY=True`
`{capture_dir}/camera_info.yaml`	YAML	Each successful capture
`{capture_dir}/capture.json`	JSON	Contains pose data, timestamps, joint state, and depth quality information
`{capture_dir}/depth_visualization.png`	PNG	Diagnostic depth visualisation output

Table 3: Files written per scan session and capture.

Four fields in `capture.json` were particularly useful for validating data quality. The `timestamp_delta_color_depth_ms` field (33.4ms in the example capture) confirmed colour and depth frames were from adjacent hardware exposures, preventing colour-geometry misalignment in reconstruction. The `valid_fraction` field (0.9217) confirmed 92% of depth pixels were valid, indicating the surface was well-suited for RGB-D capture.

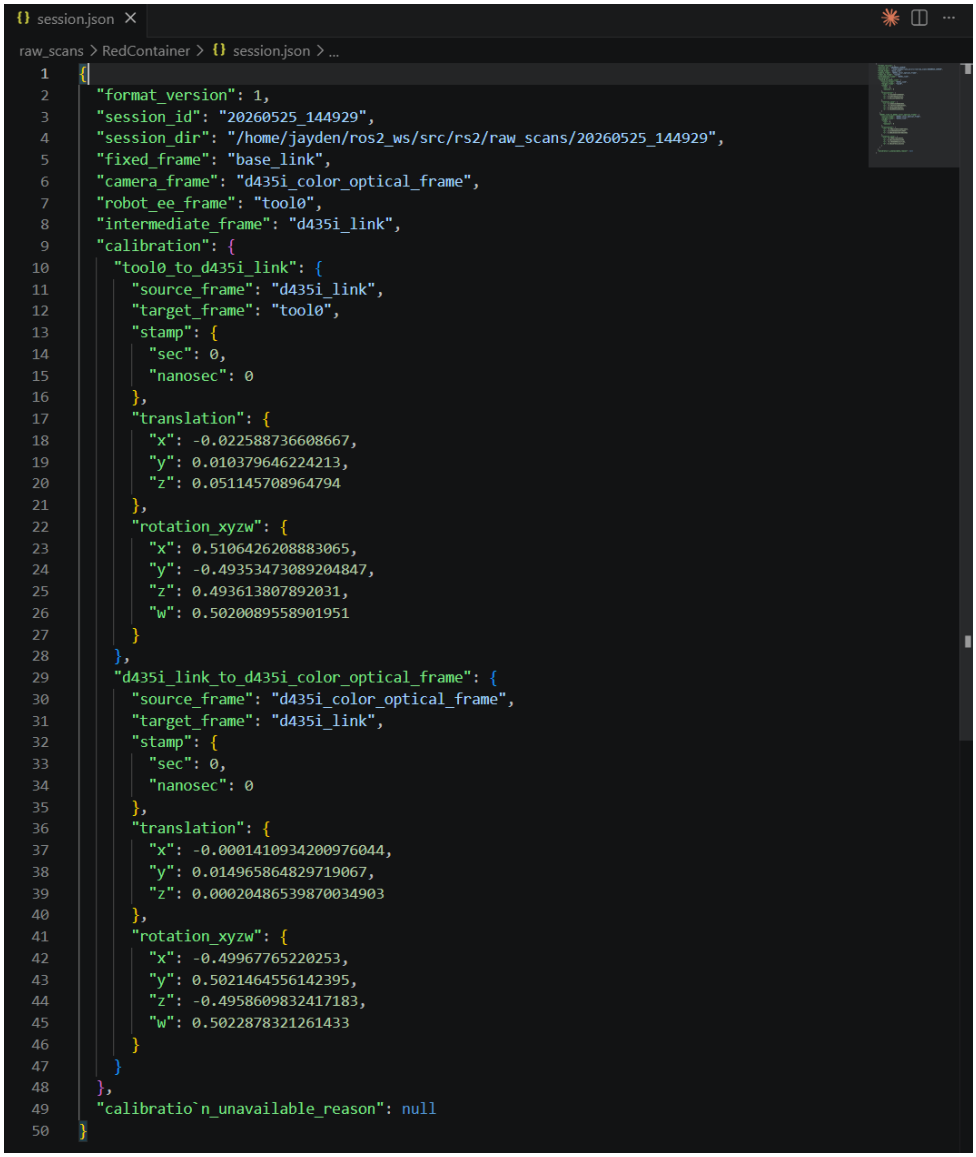


Figure 10: Individual capture metadata (`capture.json`)

The `stable` flag confirmed all six joints were below 0.01 rad/s at capture time, meaning the robot had settled before the image was taken. Finally, `pose_composed` stored the full `base_link` → `d435i_color_optical_frame` transform at the moment of capture, giving each viewpoint a self-contained pose record for reconstruction. This per-capture logging made it straightforward to identify and discard problematic captures offline — any capture with a low `valid_fraction` or large timestamp delta could be excluded from reconstruction without re-running the robot.

Artefact 2.3 — Point Cloud Workspace Boundary

Once individual captures were validated, the reconstruction pipeline needed to isolate the object from the surrounding environment. This was controlled by `config/workspace.yaml`, which defines a bounding box in the `base_link` frame and the height of the workbench surface.

The bench height (`bench.z_m = 0.000183m`) was not estimated — it was measured by running an empty scan of the workspace with no object present and fitting a horizontal plane to the resulting point cloud using RANSAC. The workbench was the only prominent flat plane within the scan volume, making the fit reliable. A 15mm margin was added above this height to absorb surface noise and reflections near the bench boundary, giving an effective `z_min` of 15.2mm. Notably, this value is never stored directly in the config — it is computed at runtime from `bench.z_m + bench.margin_m`, ensuring that re-calibrating the bench automatically updates the crop threshold without any manual adjustment.

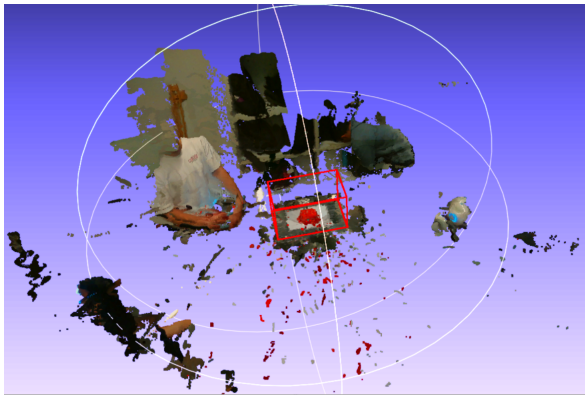


Figure 11: Full point cloud before boundary crop

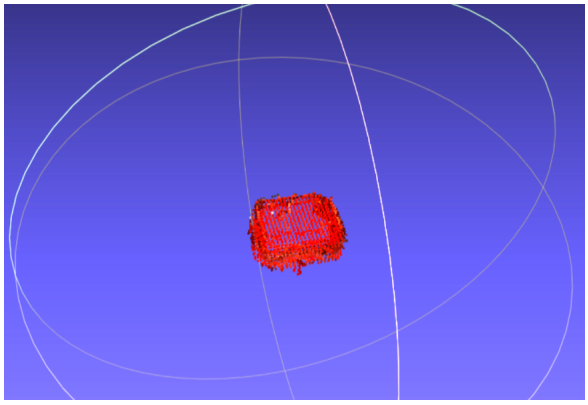


Figure 12: Point cloud after boundary crop

The before and after figures confirm the boundary correctly removed the bench surface, robot geometry, and background objects, retaining only points within the defined 50cm wide × 30cm deep × 28.5cm tall volume above the bench. This directly addresses the object segmentation test plan from the Sprint 2 presentation: the cropped output contains no non-artefact geometry, satisfying the intent of the ≥95% artefact inclusion criterion in a practical form.

Sprint 3 — System Integration

Sprint 3 marked the transition from individual subsystem development to a fully integrated system. The primary deliverable was the `main_control_node`, which acted as a centralised supervisory state machine that coordinated all three subsystems through a shared ROS2 communication architecture. Prior to this, each subsystem operated independently with no mechanism for sequencing, fault detection, or cross-subsystem synchronisation. The `main_control_node` introduced that coordination layer, enabling autonomous scan execution, heartbeat-based fault detection, and safe state transitions across the entire system.

Artefact 3.1 — Repository Structure and Commit History

Figure 13 shows a portion of the commit history during Sprint 3. The commit log reflects the iterative nature of integration work — frequent small commits as subsystem interfaces were connected, tested, and corrected. The repository structure shown in Figure 14 reflects the final organised codebase, with clear separation of responsibilities across the following main directories:

- `scripts/` — ROS2 nodes `scan_node`, `movement_node`, and `gui_node`
- `launch/ --- ROS2 launch files for simulation and real hardware`
- `helpers/` — offline reconstruction pipeline scripts
- `calibration/` — hand-eye calibration tooling and collected samples
- `config/` — workspace boundary and bench calibration parameters

This structure made it possible for team members to work on separate subsystems without conflict and for the codebase to remain navigable as complexity grew.

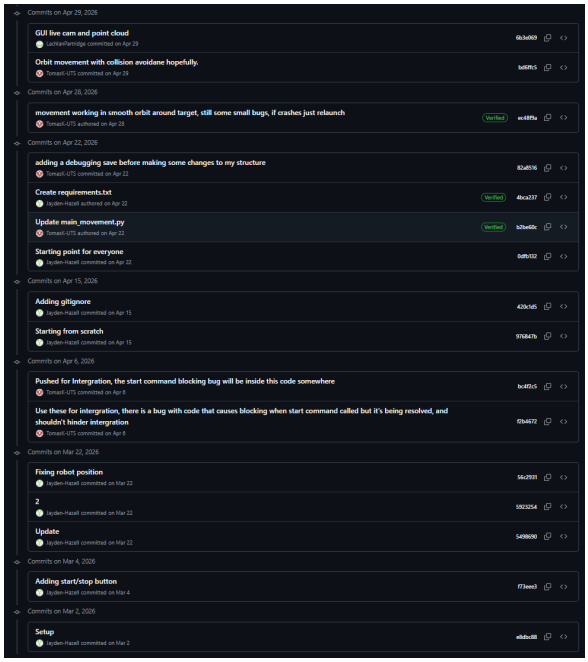


Figure 13: GitHub commit history during Sprint 3

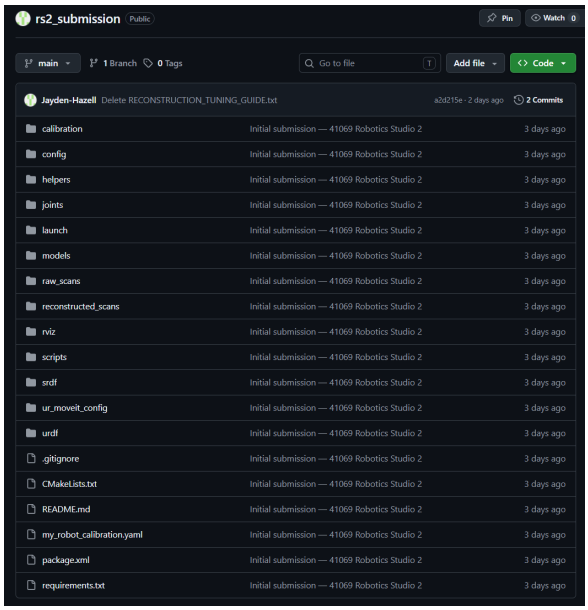


Figure 14: Repository folder structure at the end of Sprint 3

Artefact 3.2 — main_control_node State Machine

The `main_control_node` was designed around a hierarchical set of state enumerations, shown in Figure 15. Rather than managing all behaviour in a single state machine, the system separated concerns across three levels: `MainState` for top-level system modes, `AutoScanSubState` for the autonomous scan sequence, and mirrored state enumerations for the movement and scan nodes (`MovementMainState`, `ScanMainState`). This structure meant the supervisory node could monitor the internal state of each subsystem without needing direct access to their internals, it simply subscribed to their published status topics and compared against the known state enumerations.

scripts > main_control_node.py

```
31  class MainState(Enum):
32      BOOTING = auto()
33      IDLE = auto()
34      MOVE_PRE_SCAN_POSITION = auto()
35      PRE_SCAN_POSITION = auto()
36      AUTO_EXECUTION = auto()
37      MANUAL_EXECUTION = auto()
38      ERROR = auto()
39      EMERGENCY_STOP_SHUTDOWN = auto()
40  class AutoScanSubState(Enum):
41      CONFIGURING_AUTO_SCAN = auto()
42      MOVING_TO_WAYPOINT = auto()
43      CAPTURING_SCAN = auto()
44      PAUSED = auto()
45      RECONSTRUCTING_SCAN = auto()
46      SCAN_FAILED = auto()
47      SCAN_COMPLETE = auto()
48  class PendingCommandFromGUI(Enum):
49      NONE = auto()
50      MOVE_PRE_SCAN_POSITION = auto()
51      START = auto()
52      PAUSE = auto()
53  class MovementMainState(Enum):
54      BOOTING = auto()
55      IDLE = auto()
56      MOVE_PRE_SCAN_POSITION = auto()
57      PRE_SCAN_POSITION = auto()
58      AUTO_EXECUTING = auto()
59      MANUAL_EXECUTING = auto()
60      PAUSED = auto()
61      ERROR = auto()
62      UNKNOWN = auto()
63  class MovementAutoSubState(Enum):
64      NONE = auto()
65      MOVING_TO_WAYPOINT = auto()
66      WAITING_FOR_SCAN = auto()
67      FINISHED = auto()
68  class ManualMovementSubState(Enum):
69      NONE = auto()
70      IDLE = auto()
71      MOVING_TO_WAYPOINT = auto()
72      FINISHED = auto()
73  class ScanMainState(Enum):
74      BOOTING           = auto()
75      IDLE              = auto()
76      SCANNING          = auto()
77      FINISHED_SCANNING = auto()
78      RECONSTRUCTING    = auto()
79      ERROR             = auto()
80      UNKNOWN           = auto()
```

Figure 15: State enumeration definitions in `main_control_node`

Figure 16 shows the central state machine function, which runs at 10 Hz. Each tick first checks whether either the movement or scan node has entered an `ERROR` state. If so, an emergency stop is triggered immediately before any other logic executes. The heartbeat watchdog is then checked before dispatching to the appropriate state handler. This ordering is deliberate: safety checks always execute before operational logic, regardless of which state the system is in.

Figure 16: `state_machine_tick()` — central dispatch function

```
scripts > main_control_node.py
 94  class MainControlNode(Node):
880      # ----------------------------------------
881      # Heartbeat + Status Callbacks
882      # ----------------------------------------
883      def _gui_heartbeat_callback(self, _msg: Empty) -> None:
884          self.last_heartbeat_time['gui'] = self.get_clock().now()
885
886      def _movement_heartbeat_callback(self, _msg: Empty) -> None:
887          self.last_heartbeat_time['movement'] = self.get_clock().now()
888
889      def _scan_heartbeat_callback(self, _msg: Empty) -> None:
890          self.last_heartbeat_time['scan'] = self.get_clock().now()
891
892      def _all_required_heartbeats_received(self) -> bool:
893          for node_name, required in self.required_heartbeats.items():
894              if required and self.last_heartbeat_time[node_name] is None:
895                  return False
896          return True
897
898      def _heartbeat_timed_out(self, node_name: str) -> bool:
899          last_time = self.last_heartbeat_time[node_name]
900          if last_time is None:
901              return False
902
903          elapsed = (self.get_clock().now() - last_time).nanoseconds / 1e9
904          return elapsed > self.heartbeat_timeout_sec
905
906      def _check_heartbeat_watchdog(self) -> None:
907          if not self._all_required_heartbeats_received():
908              return
909
910          for node_name, required in self.required_heartbeats.items():
911              if not required:
912                  continue
913
914              if self._heartbeat_timed_out(node_name):
915                  self._trigger_emergency_stop(
916                      f'Heartbeat lost: {node_name} (>{self.heartbeat_timeout_sec:.0f}s).'
917                  )
918                  return
```

Figure 17: Heartbeat watchdog implementation

Figure 17 shows the heartbeat watchdog implementation. Each subsystem node publishes an `std_msgs/Empty` message at 2 Hz on its respective heartbeat topic. The `main_control_node` records the timestamp of each received heartbeat and checks on every tick whether any required node has exceeded the timeout threshold. If a heartbeat is lost, `_trigger_emergency_stop()` is called immediately.

```
scripts > main_control_node.py
 94  class MainControlNode(Node):
452      def _handle_auto_execution(self) -> None:
453          if self.auto_substate == AutoScanSubState.CONFIGURING_AUTO_SCAN:
454              self._handle_configuring_auto_scan()
455          elif self.auto_substate == AutoScanSubState.MOVING_TO_WAYPOINT:
456              self._handle_moving_to_waypoint()
457          elif self.auto_substate == AutoScanSubState.CAPTURING_SCAN:
458              self._handle_capturing_scan()
459          elif self.auto_substate == AutoScanSubState.PAUSED:
460              self._handle_scan_paused()
461          elif self.auto_substate == AutoScanSubState.RECONSTRUCTING_SCAN:
462              self._handle_reconstructing_scan()
463          elif self.auto_substate == AutoScanSubState.SCAN_FAILED:
464              self._handle_scan_failed()
465          elif self.auto_substate == AutoScanSubState.SCAN_COMPLETE:
466              self._handle_scan_complete()
```

Figure 18: `_handle_auto_execution()` — autonomous scan sub-state dispatcher

Figure 18 shows the `_handle_auto_execution()` function, which manages the autonomous scan sub-state machine. When the system enters `AUTO_EXECUTION`, this handler dispatches to a dedicated function for each sub-state — configuring the scan, moving to the next waypoint, capturing a scan, handling a pause, reconstructing, and completing or failing the sequence. The `MOVING_TO_WAYPOINT` and `CAPTURING_SCAN` sub-states form the core scanning loop: the robot moves to a waypoint, the movement node signals `WAITING_FOR_SCAN`, the supervisor triggers a capture, and once the scan node confirms completion the system advances to the next waypoint. This cycle repeats across all predefined positions before transitioning to `RECONSTRUCTING_SCAN`.

This design means a node that silently crashes — without publishing an error state — is detected within seconds rather than causing the system to hang indefinitely waiting for a response that will never come. It was a deliberate engineering decision to treat silence as failure rather than requiring an explicit error signal, which is a more robust assumption in a distributed ROS2 system where nodes can die without warning.

Artefact 3.3 — Boot Sequence

Before `scan_node` would accept any commands, it performed four sequential validation checks. Figure 19 shows the boot sequence logic and what happens at each step.

The fourth check — the calibration snapshot — is non-fatal. It is given ten seconds rather than five because the static transform publisher may start slightly later in the launch sequence. Failure here is recoverable because the normal scan workflow uses `pose_composed` directly from the composed TF chain that already passed in step three.

Figure 20 shows an example boot sequence including a recovery cycle. The `main_control_node` initially entered an `ERROR` state after a heartbeat timeout — the `scan_node` had completed its own boot checks and transitioned to `IDLE`, but the heartbeat signal had not yet been received within the five second window. Once the heartbeat arrived, the node automatically recovered, restarted the boot sequence, and confirmed all required nodes were `IDLE` before transitioning to `IDLE` itself. This demonstrates the resilience of the boot logic — a timing edge case that would have caused a permanent failure in a simpler implementation was handled gracefully without any manual intervention.

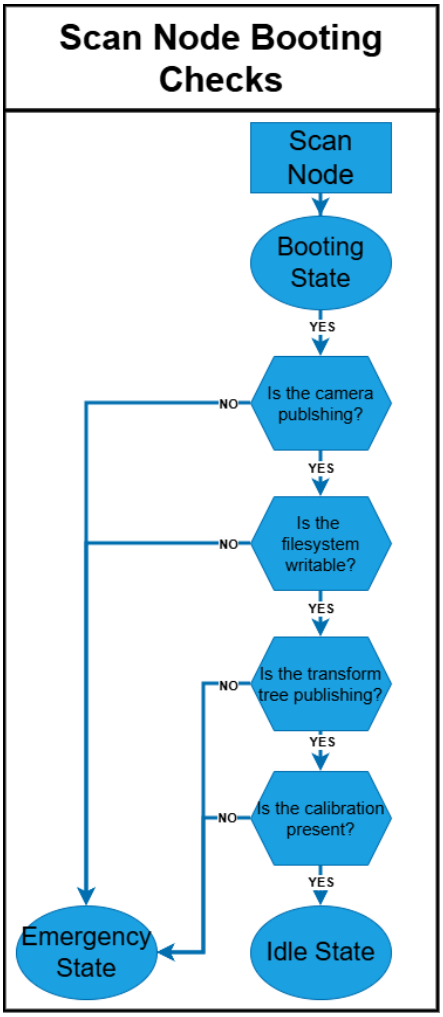


Figure 19: Boot sequence flowchart

```
[scan_node.py-13] [INFO] [1780301549.365283593] [scan_node]: Scan node started. use_fake_hardware=True
[scan_node.py-13] [INFO] [1780301549.365552240] [scan_node]: Initial state: BOOTING
[movement_node.py-12] [INFO] [1780301549.423871192] [movement_node]: Resolution updated: high. Using 32/32 scan waypoints.
[movement_node.py-12] [INFO] [1780301549.424280131] [movement_node]: Boot check passed. Loaded 32 scan waypoints from /home/jayden/ros2_ws/install/share/rs2/joints/autoScan.csv
[movement_node.py-12] [INFO] [1780301549.424505054] [movement_node]: State change: BOOTING -> IDLE
[main_control_node.py-7] [INFO] [1780301549.425278782] [main_control_node]: movement_node: IDLE
[scan_node.py-13] [INFO] [1780301549.453717839] [scan_node]: Fake hardware: skipping boot checks.
[scan_node.py-13] [INFO] [1780301549.454012347] [scan_node]: State change: BOOTING -> IDLE
[main_control_node.py-7] [ERROR] [1780301549.970134552] [main_control_node]: Boot timeout: no heartbeat from ['scan'] after 5s. Check that all required nodes started.
[main_control_node.py-7] [INFO] [1780301549.970337238] [main_control_node]: State: BOOTING -> ERROR
[main_control_node.py-7] [INFO] [1780301550.351605580] [main_control_node]: scan_node: IDLE
[main_control_node.py-7] [INFO] [1780301550.372197346] [main_control_node]: Heartbeat(s) received from ['scan'] — recovering from boot timeout error. Restarting boot sequence.
[main_control_node.py-7] [INFO] [1780301550.372458522] [main_control_node]: State: ERROR -> BOOTING
[main_control_node.py-7] [INFO] [1780301550.470709920] [main_control_node]: All required nodes IDLE — boot complete.
[main_control_node.py-7] [INFO] [1780301550.471019420] [main_control_node]: State: BOOTING -> IDLE
```

Figure 20: Terminal output showing successful BOOTING → IDLE transition with recovery

Artefact 3.4 — System Running (GUI and RViz)

Figure 21 and Figure 22 show the integrated system operating during an autonomous scan sequence. The GUI displays the system in the `SCANNING` state at 6% progress, with the system log showing the state transition history confirming the `main_control_node` is coordinating correctly. The second image shows the live camera feed displayed within the GUI, confirming the perception subsystem is actively streaming RGB data during the scan. Together these demonstrate that all three subsystems — perception, motion planning, and interaction — are communicating correctly through the `main_control_node` during live operation.

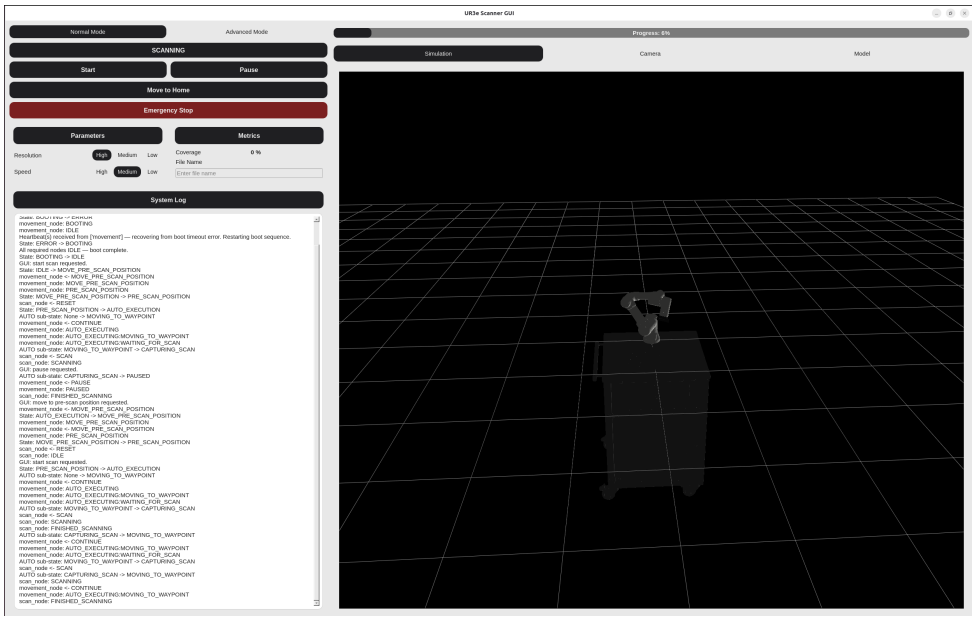


Figure 21: GUI displaying the system in `SCANNING` state at 6% progress

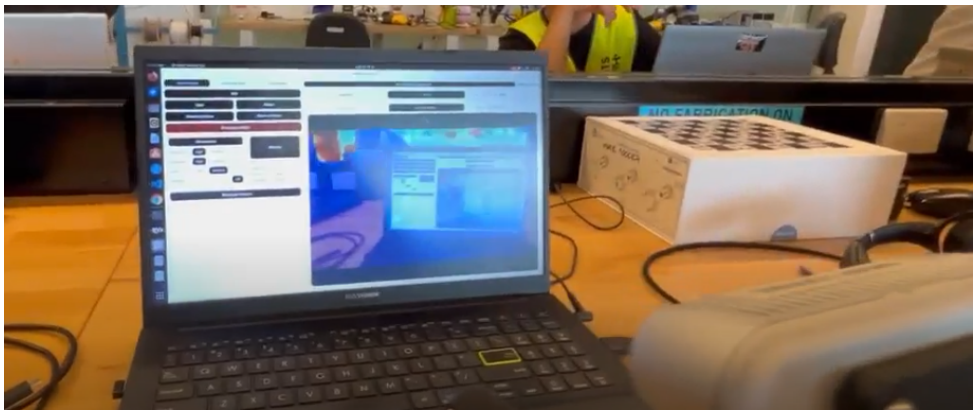


Figure 22: Live camera feed displayed within the GUI

Artefact 3.5 — Sprint 3 Video

The Sprint 3 demonstration video captures the system at a key transitional point — the three subsystems were integrated and communicating, but full autonomous operation had not yet been achieved. The video demonstrates the GUI controlling the motion and perception subsystems through a single launch file, the UR3e executing a predefined scan trajectory, and the digital twin and live camera feed updating in real time during the scan. A hand-eye calibration test is also shown, comparing the calibrated transform against the physical hardware setup, and an early point cloud reconstruction is demonstrated where data from multiple manual viewpoints is combined into a single global frame.

At this stage, snapshot triggering still required manual user input — the operator pressed a key to capture each viewpoint rather than the system triggering captures automatically. The resulting 3D model was rough, with some missing regions, but demonstrated that the robot pose and perception pipeline were correctly integrated. Full autonomous scan execution, where the robot moves through waypoints and triggers captures without any user input, was completed in the days following this video.

`https://www.youtube.com/watch?v=uiWDyrzP_Iw`

Figure 23: Sprint 3 Video Cover

The video also highlights the challenges encountered during the transition from simulation to real hardware — connectivity issues, communication debugging, and a development machine failure following a software update that required rebuilding the dual boot environment before work could continue. These setbacks are relevant context for understanding the sprint timeline and the pace of integration progress.

Sprint 4 — Optimisation and Documentation

Sprint 4 focused on refining and documenting the integrated system built in Sprint 3. The sprint addressed feedback received after the Sprint 3 presentation, optimised the scan trajectory for better object coverage, refined the reconstruction pipeline through systematic parameter tuning, and produced the technical documentation that captured all subsystem knowledge for future reference.

Artefact 4.1 — Scan Waypoint Optimisation

The original scan trajectory contained 15 manually defined waypoints that provided insufficient object coverage and included several poses at non-ideal viewing angles. To address this, a helper script `joint_recorder.py` was written, shown in Figure 24, which allowed joint positions to be recorded interactively while confirming the camera feed in the GUI showed a suitable viewing angle before committing each pose. The robot was moved around the object in freedrive mode and 32 positions were recorded covering a wider range of elevations and azimuths. This was done in collaboration with Tomas's movement node, which reads the waypoint file at launch and distributes the positions across the autonomous scan sequence.

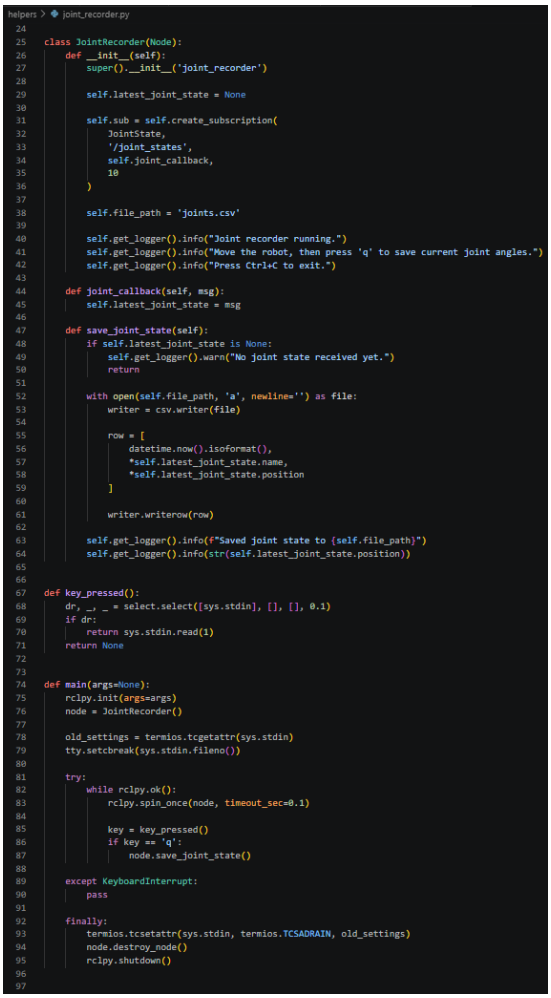


Figure 24: `joint_recorder.py` — interactive waypoint capture script

Tables 4 and 5 show the full joint configurations for the original 15-waypoint and revised 32-waypoint trajectories. All values are in radians. The 32 waypoints were recorded with intentional overlap between adjacent positions, ensuring complete object coverage and smooth robot motion between poses. This overlap also supports the resolution selection feature in the GUI — when the user selects medium or low resolution, the movement node reduces the active waypoint count from 32 to 25 or 20 respectively by evenly skipping positions, while still following a geometrically smooth path without requiring replanning.

#	**sho_pan**	**sho_lift**	**elbow**	**wrist_1**	**wrist_2**	**wrist_3**
1	-1.1766	-1.5972	-1.0361	-2.0487	1.6076	0.5084
2	0.1127	-1.8853	-1.8988	-1.9700	1.1467	3.0234
3	-0.5191	-2.3014	-0.8502	-2.3624	1.2049	2.8800
4	-0.7569	-1.9965	-0.5323	-2.4566	1.3566	2.5000
5	-0.8534	-2.5572	0.1115	3.6725	1.2845	2.8724
6	-1.3143	-2.3506	0.1202	3.7577	1.6004	3.3929
7	-1.3662	-2.4875	-0.0945	3.8994	1.6581	0.2846
8	-1.6265	-2.4588	-0.0395	3.8808	1.9369	0.8027
9	-1.6107	-2.2835	-0.0390	3.7570	1.8967	0.8708
10	-2.2403	-2.5838	-0.0372	3.6057	2.2413	0.7243
11	-1.7919	-2.2232	0.0881	3.5535	1.9985	1.3766
12	-2.4495	-1.3593	-1.8397	4.5337	2.4698	1.3765
13	-1.9079	-1.2247	-1.5629	4.5628	2.0976	2.0677
14	-1.9077	-0.7332	-2.2404	5.0673	2.1856	2.3410
15	-1.4904	-0.1142	-2.3965	4.7154	1.7782	0.2017

Table 4: Original scan trajectory — 15 waypoints (radians).

#	**sho_pan**	**sho_lift**	**elbow**	**wrist_1**	**wrist_2**	**wrist_3**
1	-1.2567	-1.3906	-1.5742	-1.5919	1.5633	0.2663
2	-0.5653	-0.9556	-2.0074	-1.6545	1.2358	1.3110
3	-0.0163	-0.9189	-2.1621	-2.0922	0.9706	2.3004
4	0.0428	-1.3489	-2.1554	-1.8876	1.0349	2.5962
5	0.1140	-1.6046	-2.1543	-1.9936	1.0003	2.7355
6	-0.1581	-2.0214	-1.6898	-2.1367	1.0104	2.8295
7	-0.4071	-2.3265	-1.1909	-2.3339	1.0067	2.8463
8	-0.7190	-2.5025	-0.7122	-2.4068	1.2402	2.9523
9	-0.8595	-2.7713	-0.2501	-2.5816	1.3730	3.0037
10	-1.0350	-2.7504	-0.2237	-2.5754	1.4693	3.0928
11	-1.3274	-2.7499	-0.1052	-2.5246	1.6502	3.3631
12	-1.3342	-2.8424	-0.1046	-2.4991	1.7687	3.3326
13	-1.6490	-2.8417	-0.1047	-2.5233	2.1013	3.6096
14	-1.8620	-2.8427	-0.1048	-2.5335	2.3021	3.6819
15	-2.1006	-2.8453	-0.1049	-2.5840	2.4448	3.6862
16	-2.4619	-2.8060	-0.1048	-2.6965	2.5200	3.6867
17	-2.4355	-2.2266	-1.5419	-1.8423	2.7389	3.6650
18	-2.6023	-1.9719	-2.1263	-1.7086	2.9538	3.6634
19	-2.5518	-1.6295	-1.9151	-1.5978	2.6078	4.2433
20	-2.1217	-1.9867	-1.9393	-1.1553	2.6104	4.3254
21	-2.0574	-2.1345	-1.7506	-1.1560	2.6145	4.3257
22	-2.0580	-1.5764	-2.2309	-0.7489	2.3041	5.1980
23	-2.1656	-1.2637	-2.2483	-1.1591	2.3854	5.1896
24	-1.8363	-1.2971	-1.9827	-1.1961	2.1084	5.3642
25	-1.4487	-1.3699	-1.7592	-1.3668	1.9017	5.8319
26	-1.2028	-1.4636	-1.6480	-1.4789	1.6011	6.2064
27	-1.9384	-1.8508	-1.4237	-1.4671	2.1985	4.5018
28	-1.6914	-2.2164	-0.6093	-2.0803	1.9438	3.9164
29	-1.3219	-2.2483	-0.6090	-2.1335	1.6981	3.3036
30	-0.7981	-2.1884	-0.5210	-2.4584	1.3528	2.6788
31	-0.3845	-1.7538	-1.3490	-2.2010	1.1515	2.2267
32	-0.2395	-1.1974	-1.9616	-1.9146	0.9822	2.2004

Table 5: Revised scan trajectory — 32 waypoints (radians).

Artefact 4.2 — System Integration Flowchart

The Sprint 3 presentation received the following feedback:

> *"Total: 9.5. Documentation (3.5): Diagrams to explain decentralised integration structure. Understanding: 6.0"*

In person, the feedback clarified that the integration concept was conveyed well but was difficult to follow without a visual representation of the full state machine logic and node coordination. In response, a full system integration flowchart was produced and included in the Sprint 4 submission and technical documentation. Figure 25 shows the completed diagram.

The flowchart captures what the Sprint 3 presentation described in words but could not show visually: the boot validation sequence for each node, the heartbeat watchdog coordination, the `AUTO_EXECUTION` sub-state loop cycling between `MOVING_TO_WAYPOINT` and `CAPTURING_SCAN`, the pause and recovery paths, and the emergency stop latching behaviour. Producing this diagram also served as a useful internal review — tracing every state transition forced a check that the implementation matched the intended design.

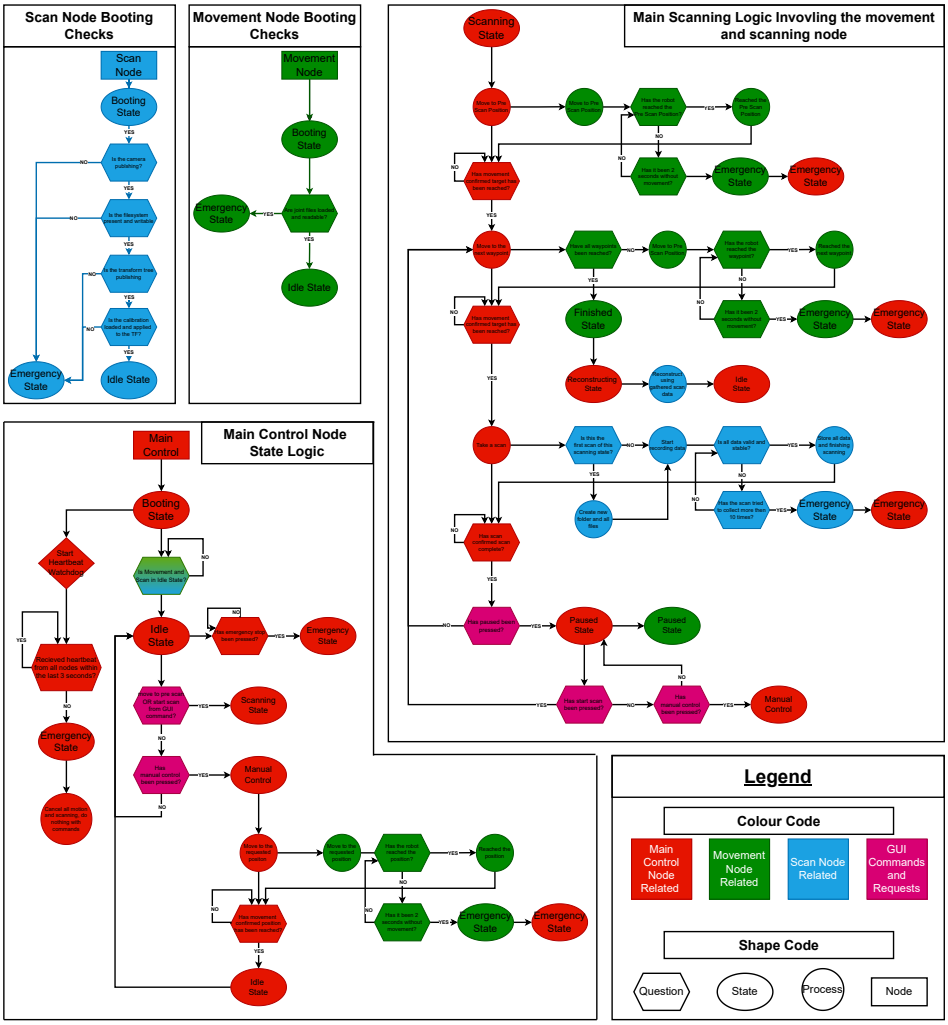


Figure 25: System integration flowchart produced in response to Sprint 3 feedback

Artefact 4.3 — Reconstruction Pipeline Tuning

The reconstruction pipeline processes raw captured scan data through two stages:
s `reconstruct.py` merges and filters the per-capture point clouds into a single aligned global cloud, and `mesh_from_cloud.py` converts that cloud into a watertight surface mesh. Both scripts expose a set of tunable constants that significantly affect output quality. Reaching a clean, usable mesh required iterative experimentation — adjusting one parameter at a time and evaluating the result visually in MeshLab.

The primary issue encountered was lumpy, bridging surfaces where the mesh tried to connect regions of the object that were only sparsely scanned. Poisson surface reconstruction works by fitting a smooth surface to the point cloud — when point density is low in a region, it invents geometry to fill the gap, producing bumpy surfaces that do not reflect the actual object shape. Table 6 summarises the key issues encountered and the parameters adjusted to resolve them.

Symptom	**Cause**	**Fix**
Lumpy, bridging surfaces	`POISSON_DENSITY_THRESHOLD` too low — Poisson invents geometry in low-density regions	Raised `POISSON_DENSITY_THRESHOLD` to cull low-confidence vertices
Floating blobs around the object	Depth noise near edges and surface reflections creating isolated clusters	Tightened `ROR_NB_POINTS` and `ROR_RADIUS_M`; increased `DBSCAN_MIN_CLUSTER_PTS`
Ragged fringe at the base	Poisson extending geometry into the bench region	Raised `bench.margin_m` in `workspace.yaml`
Over-smoothed result losing detail	`SMOOTH_PASSES` too high, rounding genuine geometry	Reduced `SMOOTH_PASSES` until sharp edges were preserved

Table 6: Reconstruction issues encountered and parameter adjustments made.

Table 7 shows the final deployed parameter values across both reconstruction scripts after tuning.

Parameter	Value	Effect
SOR_NB_NEIGHBORS	20	Neighbours used for statistical outlier removal per capture
SOR_STD_RATIO	2.0	Points beyond mean + 2σ are removed
ROR_NB_POINTS	10	Minimum neighbours within search radius to retain a point
ROR_RADIUS_M	0.010 m	Search radius for radius outlier removal
DBSCAN_MIN_CLUSTER_PTS	50	Clusters smaller than this are discarded as noise
POISSON_DEPTH	7	Octree depth controlling mesh detail level
POISSON_DENSITY_THRESHOLD	0.20	Vertices below this density percentile are removed after reconstruction
SMOOTH_PASSES	80	Taubin smoothing iterations preserving volume while reducing noise
voxel_size	0.003 m	Voxel grid downsampling resolution applied to merged cloud

Table 7: Final tuned reconstruction parameters.

Figures 26 and 27 show the raw point cloud output before any filtering and the final OBJ mesh after the full pipeline. The improvement in surface quality reflects the combined effect of the parameter tuning described above.

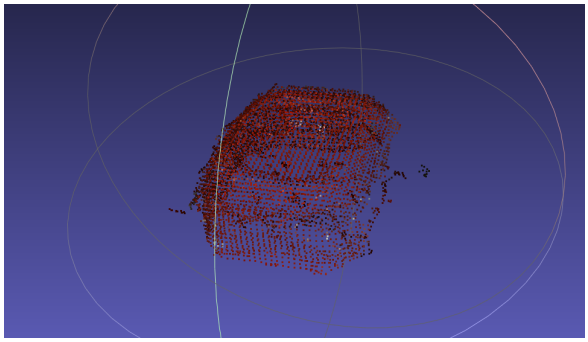


Figure 26: Raw accumulated point cloud before filtering

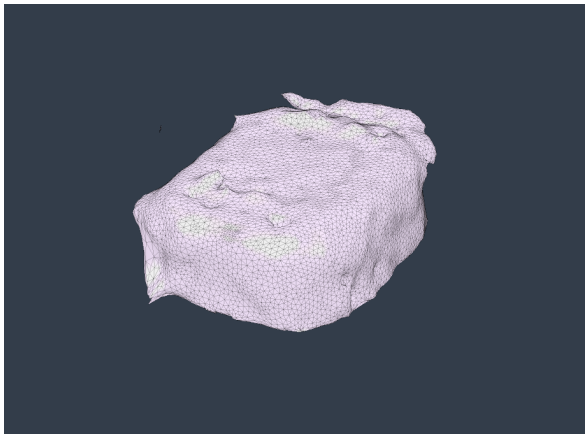


Figure 27: Final OBJ mesh after full reconstruction pipeline

Artefact 4.4 — Technical Documentation

The technical documentation was a collaborative team deliverable produced at the end of Sprint 4. My contribution covered the entire Perception and Mapping subsystem section. Figure 28 shows the table of contents for that section, which structured the knowledge into installation, configuration, testing, and reference material.

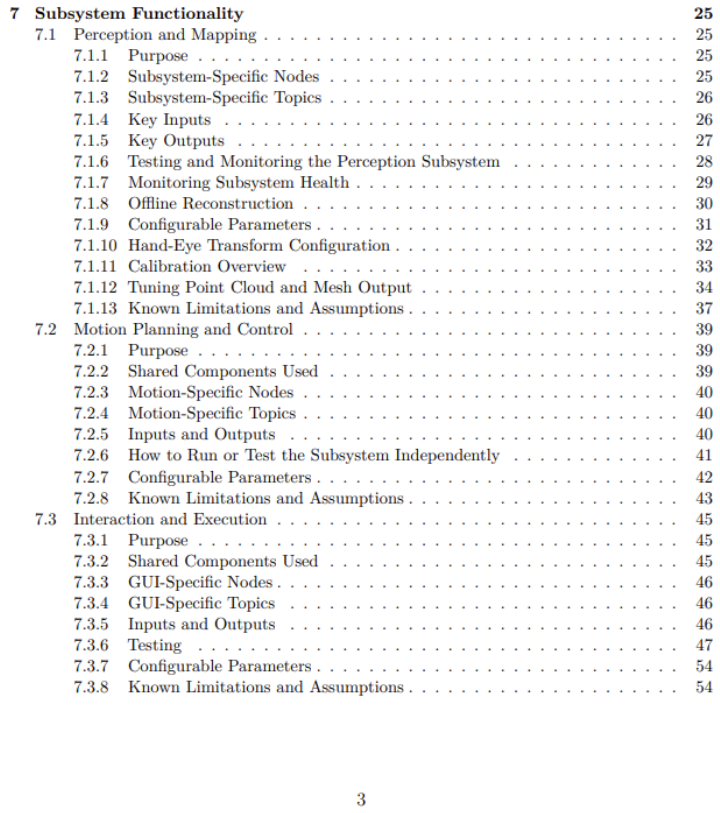


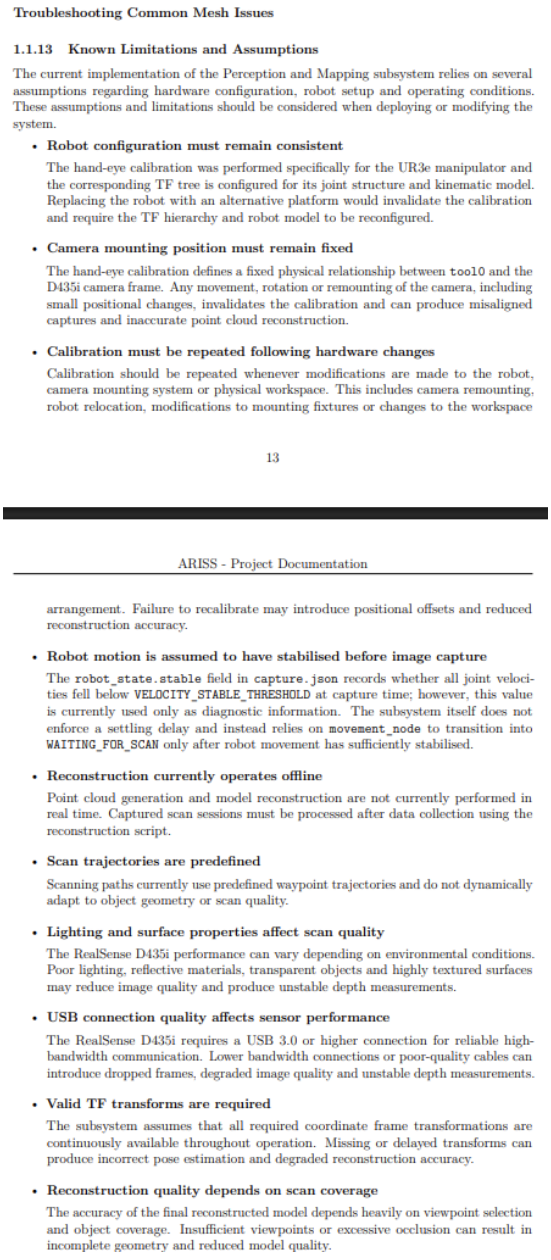
Figure 28: Table of contents for the Perception and Mapping section of the documentation

The section covered the following areas:

- **Subsystem purpose and node descriptions** — defining the role of `scan_node`, `reconstruct.py`, and `mesh_from_cloud.py` within the overall system
- **Key inputs and outputs** — documenting every ROS2 topic consumed and published, including message types and trigger conditions
- **Testing and monitoring** — step-by-step commands for verifying camera streams, scan state transitions, and heartbeat health during live operation
- **Offline reconstruction** — instructions for running the two-stage pipeline manually, including all CLI arguments and reconstruction modes (Mode A and Mode B)
- **Configurable parameters** — the full set of tunable constants for both `reconstruct.py` and `mesh_from_cloud.py`, with explanations of the effect of each value
- **Hand-eye and bench calibration** — step-by-step procedures for repeating both calibrations, including when recalibration is required
- **Known limitations and assumptions** — an honest assessment of where the subsystem breaks down, including surface reflectivity, USB bandwidth constraints, and the offline-only nature of the reconstruction pipeline

Writing the known limitations section, shown in Figure 29, was particularly useful as a reflective exercise. It required articulating not just what the subsystem could do, but where it would fail and why. Documenting that the reconstruction pipeline remained offline-only, for example, was an acknowledgement of the gap between the original D-level criteria and what was actually delivered. Having to write it down clearly — in a way that someone else could read and understand — made that gap more concrete than it had felt during development. It also clarified which limitations were fundamental constraints of the hardware (RealSense performance on reflective surfaces, USB bandwidth) versus which were engineering choices made under time pressure (offline-only reconstruction, predefined scan trajectories) that a future iteration could address.

The documentation was divided across the team by subsystem, with each member responsible for the section corresponding to their primary contribution. Reviewing Tomas and Lachlan's sections on motion planning and the GUI provided useful perspective on parts of the system that had only been understood at the interface level during development — seeing the internal logic written out made the coordination between subsystems clearer in hindsight. The cross-review process also caught several inconsistencies between subsystem descriptions, particularly around topic names and state machine behaviour, that were corrected before submission. This confirmed that documentation is not just a record of what was built, but a tool for verifying that the team's understanding of the system was actually shared rather than assumed.



Troubleshooting Common Mesh Issues

1.1.13 Known Limitations and Assumptions

The current implementation of the Perception and Mapping subsystem relies on several assumptions regarding hardware configuration, robot setup and operating conditions. These assumptions and limitations should be considered when deploying or modifying the system.

- **Robot configuration must remain consistent**
  The hand-eye calibration was performed specifically for the UR3e manipulator and the corresponding TF tree is configured for its joint structure and kinematic model. Replacing the robot with an alternative platform would invalidate the calibration and require the TF hierarchy and robot model to be reconfigured.
- **Camera mounting position must remain fixed**
  The hand-eye calibration defines a fixed physical relationship between `tool0` and the D435i camera frame. Any movement, rotation or remounting of the camera, including small positional changes, invalidates the calibration and can produce misaligned captures and inaccurate point cloud reconstruction.
- **Calibration must be repeated following hardware changes**
  Calibration should be repeated whenever modifications are made to the robot, camera mounting system or physical workspace. This includes camera remounting, robot relocation, modifications to mounting fixtures or changes to the workspace

13

ARISS - Project Documentation

arrangement. Failure to recalibrate may introduce positional offsets and reduced reconstruction accuracy.

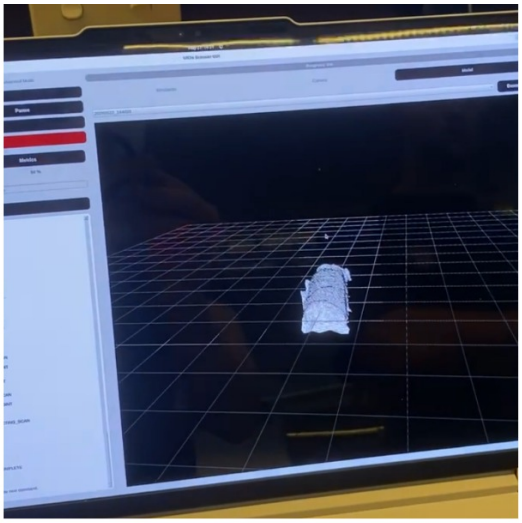
- **Robot motion is assumed to have stabilised before image capture**
  The `robot_state.stable` field in `capture.json` records whether all joint velocities fell below `VELOCITY_STABLE_THRESHOLD` at capture time; however, this value is currently used only as diagnostic information. The subsystem itself does not enforce a settling delay and instead relies on `movement_node` to transition into `WAITING_FOR_SCAN` only after robot movement has sufficiently stabilised.
- **Reconstruction currently operates offline**
  Point cloud generation and model reconstruction are not currently performed in real time. Captured scan sessions must be processed after data collection using the reconstruction script.
- **Scan trajectories are predefined**
  Scanning paths currently use predefined waypoint trajectories and do not dynamically adapt to object geometry or scan quality.
- **Lighting and surface properties affect scan quality**
  The RealSense D435i performance can vary depending on environmental conditions. Poor lighting, reflective materials, transparent objects and highly textured surfaces may reduce image quality and produce unstable depth measurements.
- **USB connection quality affects sensor performance**
  The RealSense D435i requires a USB 3.0 or higher connection for reliable high-bandwidth communication. Lower bandwidth connections or poor-quality cables can introduce dropped frames, degraded image quality and unstable depth measurements.
- **Valid TF transforms are required**
  The subsystem assumes that all required coordinate frame transformations are continuously available throughout operation. Missing or delayed transforms can produce incorrect pose estimation and degraded reconstruction accuracy.
- **Reconstruction quality depends on scan coverage**
  The accuracy of the final reconstructed model depends heavily on viewpoint selection and object coverage. Insufficient viewpoints or excessive occlusion can result in incomplete geometry and reduced model quality.

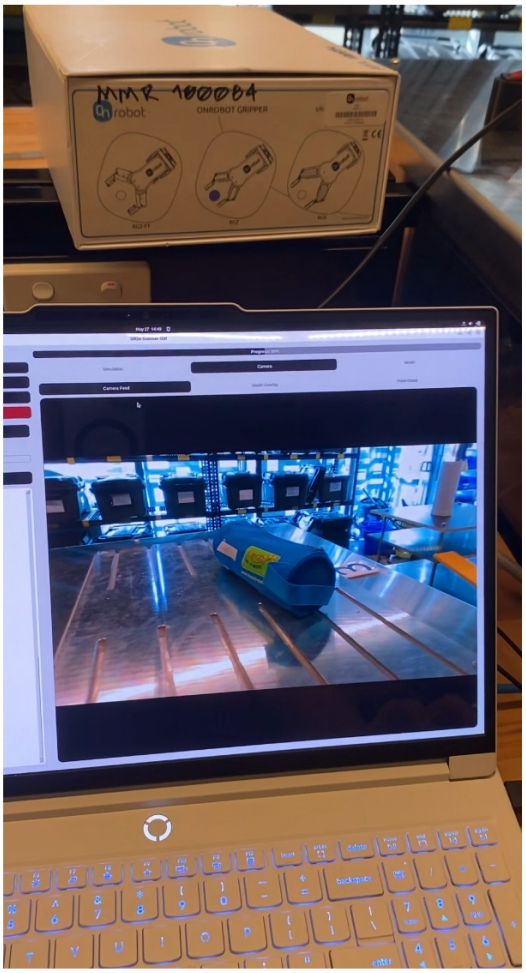
Figure 29: Known limitations and assumptions section in documentation

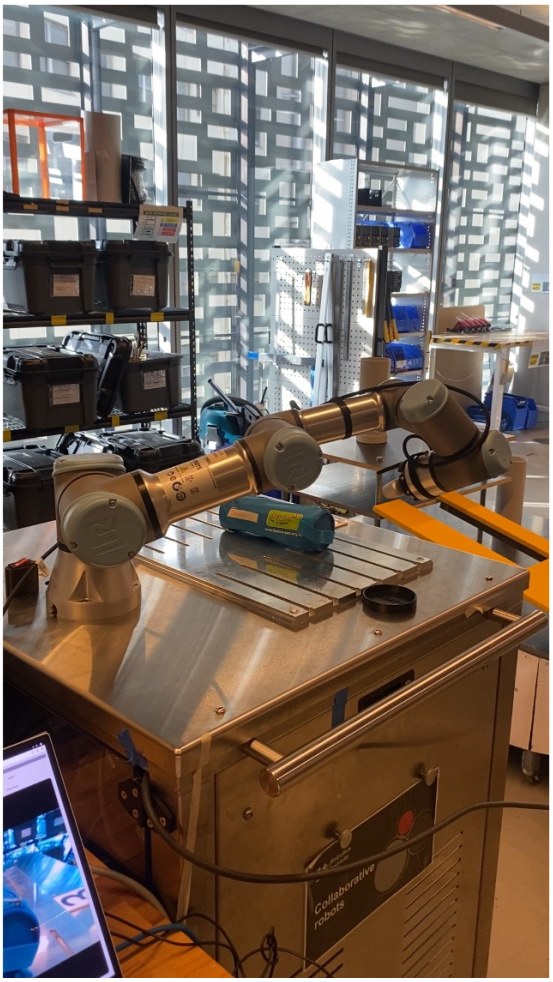
The full project repository, is publicly available at `https://github.com/Jayden-Hazell/rs2_submission`.

Demonstration Day

On demonstration day we presented to a small group of students, our client Robert Fitch and our subject coordinator Tony Le, both professors at the University of Technology Sydney. The demonstration went well, the system created a watertight mesh of a non-uniform pencil case, in a fully autonomous scan sequence.







Reflection

I really enjoy studio subjects as they guide you through a full project design from start to finish. You learn as needed along the way and develop throughout, learning how to work in a team and finishing with a full product that can include code, mechanical design, and documentation. This subject required us to develop across the following four subject learning outcomes:

1. Apply design/systems thinking to the analysis of a robotics problem.
2. Apply technical skills to develop, model and/or evaluate a robotics path planning or control solution.
3. Demonstrate effective collaboration and communication skills as an effective member or team leader of team/s.
4. Conduct critical self, peer and team reflection for performance evaluation.

SLO 1 — Apply design and systems thinking to the analysis of a robotics problem

Before any code was written, the team needed to agree on what the system had to do, who owned each piece of it, and how the pieces would connect. The project contract and system architecture diagram were the primary tools for this. Writing the Perception and Mapping subsystem criteria in Artefact 1.1, defining what the subsystem needed to produce at each grade band from Pass to Perfect, forced me to reason about the subsystem not as code to write but as a set of outputs to be produced, thresholds to be met, and interfaces to be respected. This is systems thinking at the requirement stage: understanding a component's role within a larger whole before its implementation exists.

The system architecture diagram in Artefact 1.3 extended this thinking visually, mapping the three subsystems, their internal processes, and the data flowing between them. When feedback identified that the interface definitions lacked detail, the revised diagram annotated each connection with its ROS2 topic name and message type, moving from conceptual arrows to concrete communication contracts.

The most significant systems-level decision of the project was the deliberate replacement of adaptive rescanning with advanced safety at the D-level system evaluation, discussed in Artefact 1.1. At the contract stage, adaptive rescanning was included as a D-level system capability. As the semester progressed it became clear that this feature required all three subsystems to be operating reliably in concert before the gap-filling logic could even be attempted, and that the reconstruction pipeline remaining as a standalone offline script made real-time adaptive rescanning impossible within the remaining time. The team recognised this, weighed the cost of a shallow implementation against the value of robust safety behaviour, and made a deliberate scope decision.

The same reasoning applied when the `main_control_node` was designed, documented in Artefact 3.2. Rather than allowing subsystems to communicate directly, I introduced a centralised supervisory state machine to sequence commands and monitor subsystem health, treating the system as a coordinated whole rather than three independent parts.

SLO 2 — Apply technical skills to develop, model and/or evaluate a robotics path planning or control solution

My primary technical contributions were the Perception and Mapping subsystem and the `main_control_node`, the supervisory state machine that coordinated all three subsystems during autonomous operation.

The `main_control_node`, shown in Artefact 3.2, was designed around a hierarchical set of state enumerations separating top-level system modes, autonomous scan sub-states, and mirrored state representations for the movement and scan nodes. The central design decision was to treat silence as failure: rather than requiring a node to publish an explicit error signal, the heartbeat watchdog detects any node that stops publishing within three seconds and triggers an emergency shutdown immediately. In a distributed ROS2 system where nodes can crash without warning, this is a more robust assumption than waiting for an error that may never arrive.

Hand-eye calibration, documented in Artefact 2.1, established the spatial relationship between the robot wrist and the camera. Using a ChArUco board, 22 paired observations were collected and five solvers were run in parallel. The consensus result was validated against the CAD model, and an earlier calibration attempt that appeared close, offset by only a few centimetres, was shown to produce severe ghosting in the reconstruction output. This was a concrete evaluation of calibration quality: comparing the transform against expected geometry and against the visual output it produced.

The scan session data structure, shown in Artefact 2.2, was designed to make every capture independently verifiable. Recording timestamp synchronisation, depth validity fraction, joint stability, and the full camera pose per capture meant that problematic viewpoints could be identified and excluded in post-processing without re-running the robot.

The reconstruction pipeline, detailed in Artefact 4.3, required systematic parameter tuning. Starting from a mesh with lumpy bridging surfaces and floating blobs, I adjusted each parameter in isolation and evaluated the result visually in MeshLab, raising the Poisson density threshold to remove invented geometry, tightening outlier removal to discard noise clusters, and adjusting the workspace boundary to eliminate bench surface contamination. The waypoint trajectory, documented in Artefact 4.1, was also improved from 15 to 32 positions using a helper script that allowed me to confirm each camera angle in the GUI before committing a pose, directly improving scan coverage and therefore mesh quality.

SLO 3 — Demonstrate effective collaboration and communication skills as an effective member or team leader of team/s

The team used two communication channels throughout the project. Microsoft Teams was used for file sharing and formal coordination, storing deliverables, risk assessments, and documentation drafts in a shared space accessible to all members. SMS was used for smaller, faster updates between sessions, flagging issues, confirming fixes, and coordinating before lab time. Figure 30 shows a thread where Tomas identified integration issues, I confirmed I would address them and pushed a fix shortly after, and I later commented on sections of the technical documentation to flag areas needing review.

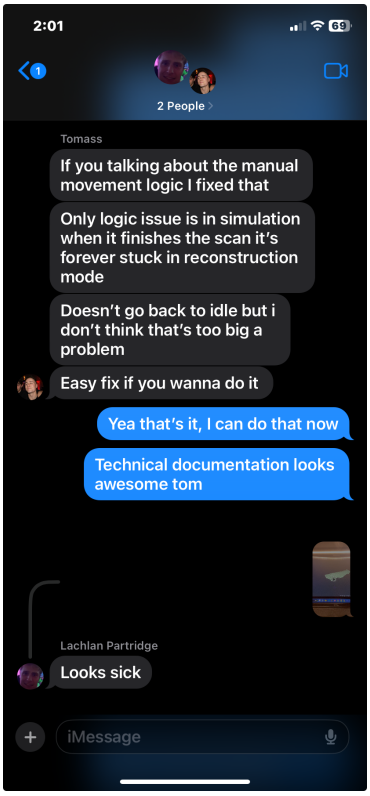


Figure 30: SMS thread showing technical issue flagging, fix confirmation, and documentation review comments

A concrete example of where communication broke down and had to be corrected involved a topic naming inconsistency: Lachlan had implemented a topic named `scan/file_name` while I had expected `gui/file_name` based on our interface conventions. We had discussed it in person but both forgot to document the decision. The mismatch was not caught until integration, where it took approximately 20 minutes of debugging to identify what turned out to be a one-line fix. This was a direct lesson in the cost of undocumented verbal agreements during integration, and it reinforced why the interface annotations added to the architecture diagram in Artefact 1.3 in response to Sprint 1 feedback mattered in practice, not just on paper.

Within the team I naturally took a team lead role, working best when setting goals and assigning responsibilities based on each person's strengths. I took ownership of the Perception and Mapping subsystem and led the development of the `main_control_node`. Tomas is thorough and methodical, making him the natural choice for documentation, video production, and final submission preparation. Lachlan's strength is in mechanical design and CAD, which drove the physical integration of the camera mount and sampling kit. This division was not formally assigned but emerged naturally from working together across multiple subjects, and it meant each person was contributing in the area where they were most effective. Figure 31 shows the full list of files shared across the semester, reflecting the breadth of collaborative deliverables produced throughout the project.

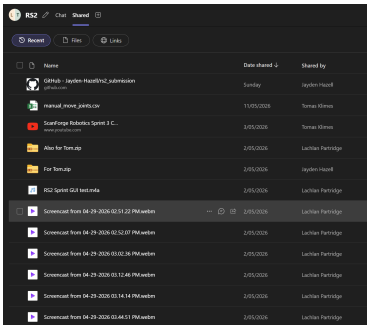


Figure 31: Shared files on Microsoft Teams throughout the project

SLO 4 — Conduct critical self, peer and team reflection for performance evaluation

The most instructive moment of the project on a personal level came during a testing session when the robot crashed into the table, breaking a cable and damaging the relationship with the lab technicians. When we reviewed what had gone wrong, it became clear that the motion planner had found an alternative Cartesian path that passed through the table surface, one that had not been caught during simulation. This led directly to three changes: switching from Cartesian to joint-space motion planning, implementing joint limits and a collision plane below which the robot could not move, and enforcing simulation of all new waypoints before running them on the physical robot, as reflected in the boot sequence safety checks shown in Artefact 3.3. The incident was frustrating at the time but produced some of the most durable engineering decisions in the project.

The calibration process, documented in Artefact 2.1, reinforced a similar lesson. The first calibration attempt appeared close to correct, a few centimetres off from the CAD reference, and was initially considered acceptable. Seeing that small error produce severe ghosting in the reconstruction output changed how I thought about precision: in a system where every capture is transformed into a global coordinate frame, a small rotational error compounds across every viewpoint. It was a lesson about the danger of treating close enough as equivalent to correct before seeing the downstream consequences.

On a peer level, Tomas and Lachlan both contributed consistently and to their strengths throughout the project. The main area where the team could have improved was earlier and more explicit documentation of interface decisions, as the topic naming incident demonstrated. That is as much a failure of my own coordination as team lead as it is a general process issue.

Overall Reflection

ARISS was the most complete project I have worked on in an academic setting, combining hardware integration, software architecture, calibration, 3D reconstruction, and team coordination into a single deliverable that worked on real hardware. The robot crashing into the table mid-semester was the moment that most clearly illustrated the difference between simulation and physical deployment. The fix required understanding not just what went wrong but why the planner chose that path, and then building constraints into the system to prevent it happening again.

If I were to approach it again, I would design the reconstruction pipeline as a ROS2 node from the start rather than as a standalone script, and I would establish a shared interface specification document before integration began rather than relying on verbal agreements that were not always carried through. The gap between a working simulation and a working physical system is not just technical. It requires a higher tolerance for unexpected failure, a discipline around documenting decisions in the moment, and a team that surfaces problems quickly rather than working around them in isolation.